

Uni[MASK]：一个掩码预测模型，做完序列决策里的所有任务

NeurIPS 2022，UC Berkeley、Microsoft Research 与 Carnegie Mellon 把行为克隆、离线强化学习、动力学建模与目标条件生成，统一成同一个掩码问题

BERT 的训练配方简单得近乎朴素：把序列里的一部分 token 随机遮住（masking），让一个双向模型把它们预测回来，就能得到可以迁移到一整类下游任务的表示。正是这个想法，重新组织了自然语言处理的大半江山。这篇论文提出了一个很直接的问题：既然序列决策里的轨迹本质上也是序列，同样的技巧能不能重新组织我们为它建模的方式？

在强化学习和模仿学习里，一条轨迹就是一串状态、动作和回报（returns）。可是我们通常会为每一个想问的问题单独训练一个模型：一个网络做行为克隆（behavior cloning），另一个做离线强化学习（offline RL）与回报条件生成，再一个做前向或逆向动力学，还有一个做目标或路点条件生成。这些模型读取的其实是同一个对象，却各自孤立地搭建和训练，把任务之间共享的结构白白丢掉了。

Uni[MASK] 从一个朴素的观察出发：上面这些问题，无非是同一条轨迹上的不同掩码，露出一部分 token、预测另一部分。这样一写，这些任务就不再是彼此独立的问题，而是对同一个模型的不同查询。一个双向 transformer，只要在很多种掩码模式上训练，就能回答所有这些问题；而实验也表明，它常常能匹配甚至超过它想要替代的那些单任务专用模型，并在稍加微调（fine-tune）之后稳定地胜过它们。

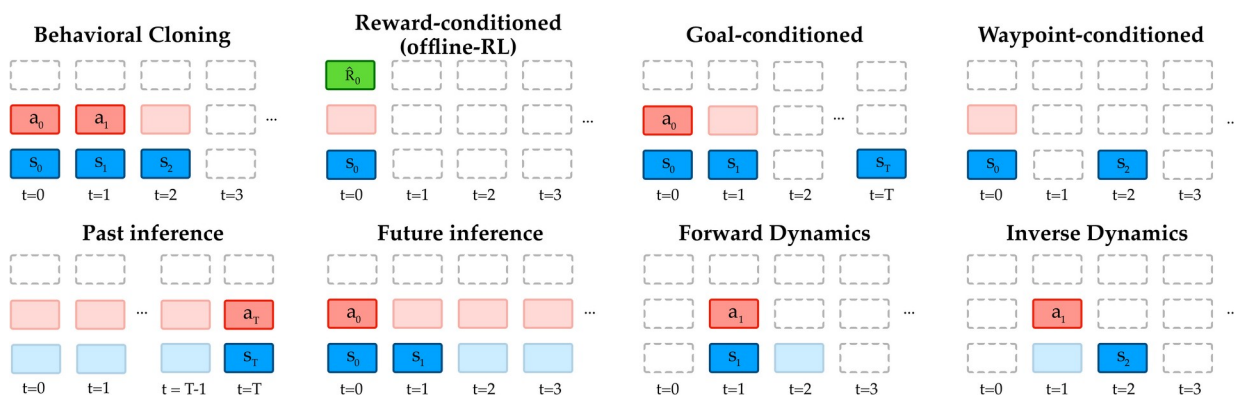


图 1：把常见的决策任务写成同一条轨迹上的掩码方案。实色 token 是喂给模型的输入，半透明 token 是模型需要预测的输出。行为克隆、回报与目标条件生成、路点条件生成、前向与逆向动力学，区别只在于遮住了哪些状态、动作与回报 token。

论文：<https://arxiv.org/abs/2211.10869> · 代码：<https://github.com/micahcarroll/uniMASK>

为同一条轨迹养了一整个模型动物园

先想想，面对一批记录下来的轨迹，实践者可能会提出哪些需求。行为克隆问的是：给定至今看到的状态和动作，下一个动作是什么？离线强化学习的回报条件形式，问的是给出一个能达到目标回报的动作。前向动力学从当前状态和动作预测下一个状态；逆向动力学则从相邻两个状态反推出连接它们的动作。目标条件与路点条件生成，要求生成经过指定未来状态的行为。这些听上去像是不同的研究方向，各有各的架构和训练流水线，但它们操作的都是同一条由状态、动作、回报组成的轨迹。

把它们分开处理，代价并不只是工程上的重复。一个只为行为克隆训练的模型，学不到一个好的动力学模型本可以教给它的环境知识；而一个动力学模型，也享受不到目标条件生成所刻画的行为结构。如果这些任务确实共享表示，那么孤立地训练它们，就等于把价值留在了桌上没有拿走。

每个任务，都只是一种掩码方案

Uni[MASK] 按时间步逐步地把轨迹切成 token。每个时间步贡献一个状态 token、一个动作 token，以及一个可选的属性 token，比如回报到终点（return-to-go），也就是从这一步往后所有奖励的总和。一种掩码方案独立地规定两件事：模型能看到哪些 token 作为输入，以及它必须预测并被打分的是哪些被遮住的 token。不同方案对应不同任务：露出过去的状态和动作、预测下一个动作，就是行为克隆；再额外露出一个未来状态，就是目标条件生成；露出回报到终点 token，就是回报条件的离线强化学习。模型本身在这些任务之间毫无变化，变的只有掩码。

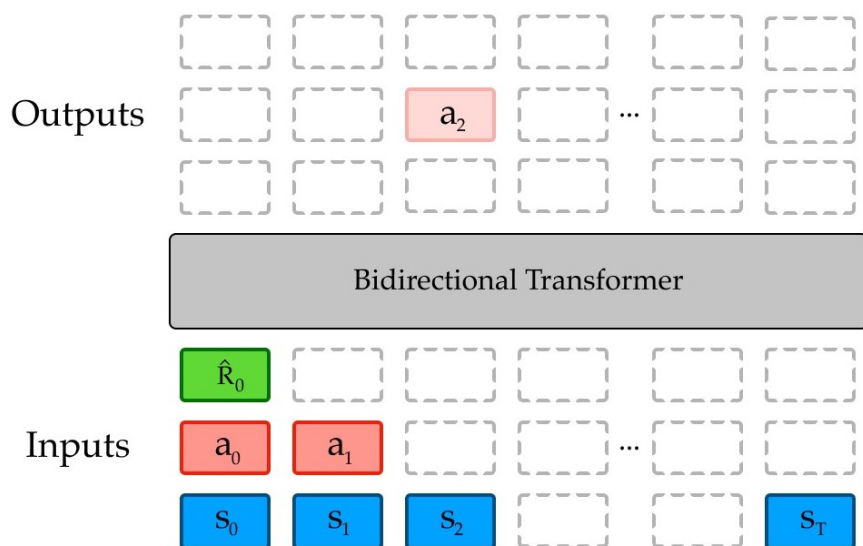


图 2：骨干是一个 BERT 风格的双向 transformer。每个时间步的状态、动作、属性 token 被堆叠成一个输入，模型一次性读入所有可见 token，并预测被遮住的输出，这里预测的是：给定过去的状态、过去的动作以及初始回报到终点时，下一个动作是什么。

由于编码器是双向的，它可以基于窗口里任意位置的 token 进行条件推断，无论过去还是未来，而这恰恰是目标条件生成、反向推断这类任务所需要的。这也是它与从左到右的 GPT 风格

模型最关键的分野，正是双向性让一个网络能够表达这一整族推断，而不只是其中因果方向的那部分。

训练同一个模型的四种方式

有了一个能表达所有任务的模型，还剩下怎么训练它的问题。论文比较了四种训练方式。单任务（single-task）在一种掩码方案上训练一个模型，也就是经典基线。多任务（multi-task）为每一段轨迹片段随机采样一种预定义的任务掩码。随机掩码（random-mask）更进一步，直接遮住任意 token，压根不锁定某个具名任务。微调（finetune）则先用随机掩码预训练，再在某个具体任务上微调。这套对比经过精心设计，让每种方式都单独隔离出一个因素的作用，而其中最值得盯住的，是微调。

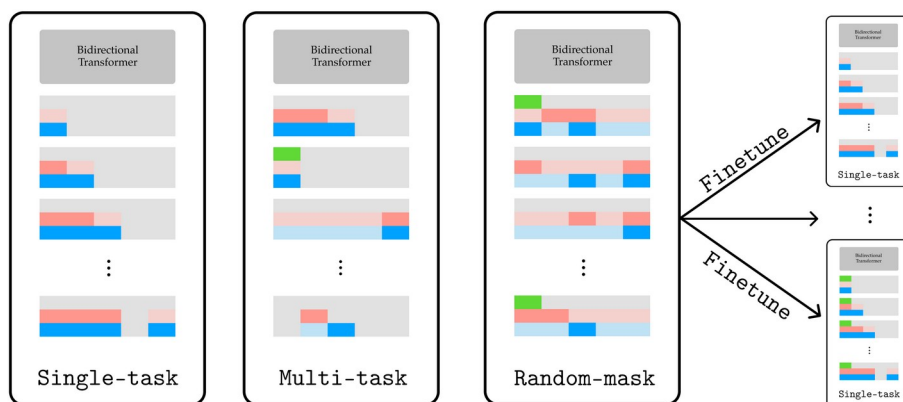


图 3：四种训练方式。单任务与多任务在具名任务掩码上训练；随机掩码在任意 token 掩码上训练；微调方式先用随机掩码预训练，再专门化到单个任务。

一个模型，许多行为

为了让框架变得可感，作者转向 MiniGrid，一个小型网格世界：智能体要先捡起钥匙、打开一扇上了锁的门，再到达目标。随后，他们用不同的掩码去查询同一个以随机掩码训练出来的模型，每一种掩码都从同一套权重里牵引出一种不同的行为。

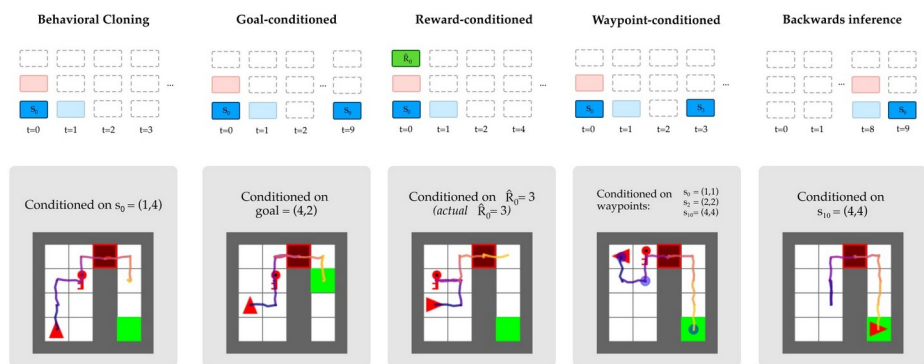


图 4：同一个随机掩码模型，五种不同的查询。从左到右依次是：从初始状态出发的行为克隆、到达一个替代目标、命中一个目标回报、穿过指定的路点，以及反向推断出一段以给定终态收尾的合理历史。轨迹带有轻微抖动以便看清。

这就是重新表述带来的定性收益。到达一个替代目标、遵守路点，或是重建一段合理的过去，并不是把几个模型拼装在一起，而是同一个模型在回答不同的掩码查询。这个框架甚至能推广到任意的属性标签，比如以一条轨迹是否曾经过网格某个角落作为条件。

数字怎么说

定性演示很有说服力，但真正要让这套重新表述立住脚的，是定量比较。在 MiniGrid 上，作者测量了各任务的验证损失，结果在 6 个随机种子上平均，并按列归一化，使每一列里最好的方法等于 1。有两个规律格外突出。用随机掩码训练，在每一个任务上都胜过单任务训练，也就是说，一个通才竟然全面压过了各路专才。而随机掩码预训练加上微调，整体表现最好，除了行为克隆之外，在每一个任务上都胜过单任务模型。

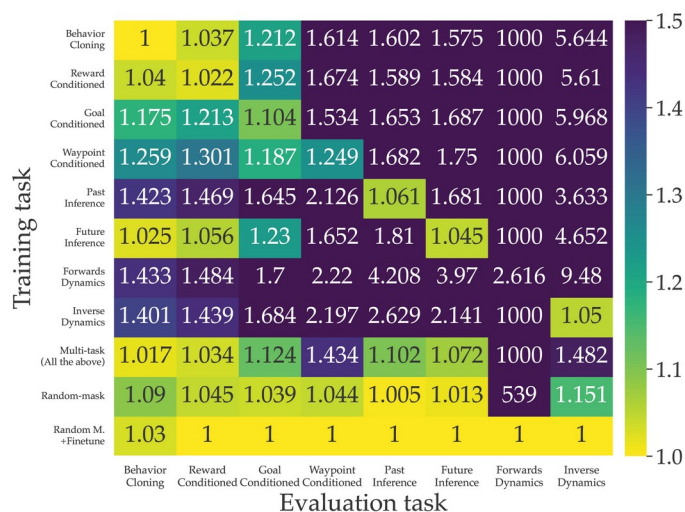


图 5：MiniGrid 上各任务的验证损失，按列归一化，使每个评测任务里最好的方法等于 1，数值越低越好。最下面一行（随机掩码预训练加上逐任务微调）在所有列上都是最好或并列最好；而靠上的单任务各行，对它们没有训练过的任务泛化得很差。

更难的考验是 Maze2D，一个来自基于 MuJoCo 的 D4RL 基准的连续控制迷宫，用测试时回报来评测，在 5 个种子上跑 1,000 次 rollout 取平均。在这里，微调不只是有帮助，而是决定性的。在上下文长度为 5 时，微调后的 Uni[MASK] 模型在行为克隆和回报条件生成上都达到约 2.73 的回报，而 Decision Transformer 只有 1.13 和 1.49，一个普通前馈网络则更低。先广泛地训练、再做专门化，胜过一开始就狭窄地训练。

表 1：Maze2D 在上下文长度 5 时的测试时回报（越高越好）。

方法	行为克隆	回报条件生成
Uni[MASK] (预训练 + 微调)	2.73	2.73
Decision Transformer	1.13	1.49

回报在 5 个种子、1,000 次 rollout 上取平均。

这组比较也坦率地暴露了一个局限。为了单独隔离骨干网络的影响，作者让单任务 Uni[MASK] 与 Decision-GPT 对垒，后者是他们提出的一个改进版 GPT 风格基线，两者唯一真正的区别就是 BERT 风格还是 GPT 风格的架构。在上下文长度为 5 时，BERT 风格模型表现不错；但到了上下文长度 10，它开始退化，被 Decision-GPT 反超，这正是 BERT 类模型在较长序列生成上众所周知的困难。

结论与边界

留下来的核心想法很干净：被这个领域当作彼此不同的许多任务，包括行为克隆、离线强化学习、动力学、目标与路点条件生成，其实都只是同一条轨迹上的不同掩码，因此一个掩码预测模型就能替代一整个专用模型的动物园。而在许多掩码上训练、再做微调，通常也胜过只在任何单个任务上训练，哪怕你其实只在乎那一个任务。

至于要注意的边界，与其说是概念上的，不如说是架构上的。BERT 风格的双向模型在较短上下文长度上很出色，却在较长序列的生成上吃力，而这正是 GPT 风格的 Decision-GPT 更占优的地方。作者也据此点出了一个自然的下一步：把 GPT 风格的骨干与随机掩码结合起来，既保留这种统一，又找回强的长程生成能力。